

Table des matières

1. Pour commencer
2. Lecture d'un niveau
3. Le retour des expressions
4. Le retour de la table des symboles

1. Pour commencer

Dans le TP précédent, nous nous sommes intéressés à mettre en mémoire les expressions (entières ou booléennes). Nous en aurons besoin pour [le projet](#) pour mettre en mémoire le code analysé et l'exécuter plus tard.

Dans ce TP, nous allons tenter de reconnaître le langage du projet, étape par étape (n'oubliez pas qu'il est possible de travailler par partie sur [le projet](#) pour tester des sous-parties de la grammaire et que l'on peut ensuite tout regrouper).

2. Lecture d'un niveau

Nous allons écrire une grammaire permettant de reconnaître un fichier contenant des niveaux. La structure de base d'un niveau est la suivante :

```
level
end
```

1. Écrivez un analyseur *Lex&Yacc* permettant de reconnaître des niveaux (vides, pour le moment). Le nom du fichier à lire est passé en argument de l'exécutable. Il doit y avoir au moins un niveau ; dans le cas contraire, cela doit générer une erreur.
2. Modifiez votre *Lex* et la fonction *yerror* pour que le numéro de ligne soit affiché lorsqu'une erreur est rencontrée.
3. Testez sur les fichiers d'exemple (accessible sur le dépôt *gitlab*) *exo1_1A.txt*, *exo1_1A.txt* et *exo1_1C.txt* : le premier doit générer une erreur, les deux autres doivent être analysés avec succès.

L'objectif de l'analyseur est de créer un niveau à chaque fois que la règle correspondante est réduite. Pour le moment, nous pouvons utiliser la structure *level_t* fournie avec le sujet du projet, avec les fonctions associées (vous pouvez modifier la structure et les fonctions à votre guise). Dès qu'un niveau est terminé, on l'affiche à l'écran.

Attention : on ne peut pas utiliser le même nom pour une variable et un symbole de la grammaire. En effet, ces derniers sont transformés en constantes lors de la compilation par *yacc*.

4. Quand doit-on créer / initialiser / afficher le niveau (la variable de type *level_t*) ? Modifiez la décoration de votre grammaire.

Si votre analyseur est correct, pour chaque niveau reconnu, il est affiché dans le terminal (pour le moment, seul le cadre formé d'étoiles est affiché). Nous allons ajouter la possibilité d'ajouter des blocs avec la méthode *put*. Elle prend en paramètres les coordonnées (x, y) et le bloc. Pour ce dernier, on a besoin des constantes suivantes :

- *EMPTY*, *BLOCK*, *TRAP*, *LIFE*, *BOMB*
- *DOOR(X)* où *X* est un numéro compris entre 1 et 99
- *ENTER* et *EXIT*
- *LADDER*
- *ROBOT*, *PROBE*
- *KEY(X)* et *GATE(X)* où *X* est un numéro compris entre 1 et 4

Une règle est nécessaire pour reconnaître tous les blocs et il faut les passer ensuite pour que la fonction *put* soit en mesure d'ajouter le bon bloc. Or un bloc est caractérisé par son type, mais également par une valeur (pour *DOOR*, *KEY* et *GATE*).

5. Créez une structure pour représenter un bloc (vous pouvez utiliser une énumération pour les types), par exemple *block_t*.
6. Ajoutez les symboles nécessaires dans *Yacc* (à l'aide de *%token*) et ajoutez les règles correspondantes pour *Lex* (une par mot-clé, le modèle est simplement une chaîne de caractères).
7. Ajoutez une règle pour reconnaître un bloc ; on retourne un *block_t*. N'oubliez pas la gestion des erreurs pour les numéros des portes, des clés et des portails.
8. Un niveau contient entre les deux mots-clés une liste d'instructions potentiellement vide.
9. Pour le moment, une instruction peut être uniquement un appel à *put*. Elle ajoute simplement le bloc dans le niveau. Attention aux coordonnées.
7. Vérifiez à l'aide des fichiers d'exemple *exo1_2*.txt*.

Vous êtes maintenant en mesure de créer des niveaux complets.

3. Le retour des expressions

Dans le TP précédent, nous avons utilisé les expressions que nous avons couplé aux arbres. Dans cette partie, nous allons les fusionner pour spécifier une expression à la place des coordonnées **x** et **y** dans la procédure **put**.

1. Ajoutez les règles associées aux expressions : n'oubliez pas de modifier votre fichier *Lex*, d'ajouter les règles de précedence pour les opérateurs et d'ajouter l'arbre comme nouveau type dans **%union**.
2. Modifiez la règle de **put** pour accepter les expressions pour **x** et **y**.

Vous devez évaluer les deux arbres pour récupérer les valeurs de **x** et **y**. Supprimez-les ensuite pour éviter les fuites de mémoire. Pour le moment, ce n'est pas pratique, mais par la suite, nous devons conserver ces arbres pour une évaluation future...

3. Procédez de même pour les règles concernant les clés, les portes et les portails.

La fonction **get** permet de récupérer le bloc situé à la position données.

4. Ajoutez une règle pour prendre en compte cette fonction.

4. Le retour de la table des symboles

Nous souhaitons ajouter la possibilité de créer des variables dans un niveau (entre les mots-clés `level` et `end`). Nous allons gérer une table des symboles locale à chaque niveau. Pour rappel, une variable n'a pas à être déclarée et comme en PHP, il n'y a pas de notion de portée locale au sein d'un bloc (une variable déclarée dans une boucle est accessible après celle-ci).

1. Où peut-on créer la variable contenant la table ? Quand doit-on l'initialiser et la vider/supprimer ?

Spoil alert

2. Déclarez la table des symboles (dans le *Yacc*, par exemple). N'oubliez pas les inclusions du fichier d'en-tête et de déclarer la table (avec `extern`) dans le *Lex*.

Il existe plusieurs solutions pour gérer les variables.

La première solution : lorsque *Lex* trouve un identificateur (une suite de caractères), il le recherche dans la table des symboles. Si aucune variable ne correspond, il renvoie le symbole `id`, sinon il renvoie le symbole `variable`. Du côté de *Yacc*, on peut rencontrer des variables dans une expression et une instruction peut être soit l'affectation d'une variable existante, soit une nouvelle variable.

La deuxième solution est de laisser *Lex* envoyer systématiquement le symbole `id` et on laisse *Yacc* faire la recherche dans la table des symboles. Dans ce cas, c'est lui qui doit être en mesure de détecter les erreurs.

3. Suivant la solution choisie, modifiez votre *Lex* pour qu'il détecte les identificateurs : une lettre de l'alphabet (minuscule ou majuscule) suivie potentiellement de lettres, de chiffres ou de `"_"`.
4. Modifiez votre structure d'arbre pour accepter comme nœud un pointeur vers un symbole. Lors de l'évaluation d'un arbre, on utilise simplement la valeur du symbole.
5. Ajoutez la règle dans l'expression pour reconnaître une variable.
6. Ajoutez maintenant les deux instructions possibles : l'affectation d'une variable et l'initialisation d'une variable.
7. Vérifiez sur les fichiers `exo4_*.txt` que tout est correct.